ApexaiQ Technology Asset Management Documentation

Weekly Research Report
Lokesh Dilip Bandurkar | Date : 09 Mar 2026

---

1. Variables

Variables are used to store data values in a program. In JavaScript, variables can be declared using **var, let, and const**.

- **var** – Function scoped (older way)
- **let** – Block scoped and commonly used
- **const** – Used for values that should not change

Example Program

```
var name = "Lokesh";
let age = 20;
const country = "India";

console.log("Name:", name);
console.log("Age:", age);
console.log("Country:", country);
```

---

2. Data Types

JavaScript supports different types of data that can be stored in variables.

❖ Common Data Types

- String
- Number
- Boolean
- Null
- Undefined
- Object
- Array

➢ **Example Program**

```
let name = "Lokesh";      // String
let age = 20;             // Number
let isStudent = true;     // Boolean
let city = null;          // Null
let salary;               // Undefined

console.log(typeof name);
console.log(typeof age);
console.log(typeof isStudent);
console.log(typeof city);
console.log(typeof salary);
```

---

3. Loops (All Types)

Loops are used to repeat a block of code multiple times.

❖ For Loop

```
for (let i = 1; i <= 5; i++) {
   console.log("Number:", i);
}
```

❖ While Loop

```
let i = 1;
while (i <= 5) {
   console.log("While Loop:", i);
```

```
  i++;
}
```

❖ Do-While Loop

```
let j = 1;

do {
  console.log("Do While:", j);
  j++;
} while (j <= 5);
```

❖ For...of Loop

```
let fruits = ["Apple", "Mango", "Banana"];

for (let fruit of fruits) {
  console.log(fruit);
}
```

❖ For...in Loop

```
let person = {
  name: "Lokesh",
  age: 20
};

for (let key in person) {
  console.log(key + ":", person[key]);
}
```

---

4. Functions (All Types)

Functions are reusable blocks of code used to perform specific tasks.

❖ Normal Function

```
function greet(name) {
  return "Hello " + name;
}

console.log(greet("Lokesh"));
```

❖ Function Expression

```
const add = function(a, b) {
   return a + b;
};

console.log(add(5, 3));
```

❖ Arrow Function

```
const multiply = (a, b) => a * b;

console.log(multiply(4, 5));
```

❖ Callback Function

```
function processUser(name, callback) {
   console.log("Processing user:", name);
   callback();
}

processUser("Lokesh", function() {
   console.log("Callback executed");
});
```

---

5. Arrays (Methods / Properties)

Arrays are used to store multiple values in a single variable.

❖ Example Program

```
let numbers = [10, 20, 30, 40];

numbers.push(50);     // Add element
numbers.pop();        // Remove last element
numbers.shift();      // Remove first element
numbers.unshift(5);   // Add element at start

console.log(numbers);
```

❖ Important Array Methods

```
let arr = [1,2,3,4,5];

arr.forEach(num => console.log(num));
```

```
let doubled = arr.map(num => num * 2);
console.log(doubled);

let filtered = arr.filter(num => num > 2);
console.log(filtered);

let sum = arr.reduce((acc, num) => acc + num, 0);
console.log(sum);

console.log(arr.includes(3));
console.log(arr.indexOf(4));
```

---

6. Objects (Methods / Properties)

Objects store data in **key-value pairs**.

❖ **Example Program**

```
let student = {
   name: "Lokesh",
   age: 20,
   course: "CSE"
};

console.log(student.name);
console.log(student.age);
```

❖ **Object Methods**

```
let user = {
   name: "Lokesh",
   greet: function() {
      console.log("Hello " + this.name);
   }
};

user.greet();
```

❖ **Object Properties / Methods**

```
let obj = {a:1, b:2, c:3};

console.log(Object.keys(obj));
```

```
console.log(Object.values(obj));
console.log(Object.entries(obj));
```

---

7. DOM and Event Handling (Callback)

DOM (Document Object Model) allows JavaScript to interact with HTML elements.

❖ **Example Program**

```
<button id="btn">Click Me</button>

<script>
let button = document.getElementById("btn");

button.addEventListener("click", function() {
  alert("Button Clicked!");
});
</script>
```

In this example, the **event listener uses a callback function** that runs when the button is clicked.

---

8. Promises

Promises are used to handle asynchronous operations in JavaScript.

❖ **Example Program**

```
let promise = new Promise(function(resolve, reject) {

  let success = true;

  if(success){
    resolve("Operation Successful");
  } else {
    reject("Operation Failed");
  }

});

promise
```

```
.then(result => console.log(result))
.catch(error => console.log(error));
```

---

9. Async and Await

Async and Await provide a cleaner way to work with asynchronous code.

❖ **Example Program**

```
function fetchData() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("Data received");
    }, 2000);
  });
}

async function getData() {
  let result = await fetchData();
  console.log(result);
}

getData();
```

This allows asynchronous code to be written in a style similar to synchronous code.

---

10. Event Loop (Research Only)

The **Event Loop** is a core concept in JavaScript that enables non-blocking asynchronous behavior.

JavaScript runs on a **single-threaded runtime**, meaning it can execute one task at a time. However, the event loop allows JavaScript to handle asynchronous operations such as API calls, timers, and file operations efficiently.

Components of Event Loop

1. **Call Stack** – Executes functions.
2. **Web APIs** – Handles asynchronous tasks like timers and network requests.
3. **Callback Queue** – Stores callbacks waiting to be executed.

4. **Event Loop** – Moves tasks from the callback queue to the call stack when it becomes empty.

This architecture allows JavaScript applications to remain responsive even when performing asynchronous operations.